

Acme Cloud Storage — New Engineer Onboarding Handbook

Welcome to the Storage Platform team. This handbook covers what you need during your first two weeks: account setup, local development, and how the on-call rotation works. Read it alongside the architecture overview document, which describes how the system fits together.

Week one: accounts and access

On your first day, your manager will sponsor your access requests for the internal developer portal, the source code repositories, and the staging cluster. Most access grants complete within a few hours, but cluster access can take up to one business day because it requires a security review. While you wait, set up your laptop using the bootstrap script in the team repository's tools directory.

You will also be issued a personal API key for the staging environment. Treat it like a password: do not commit it to source control, and rotate it immediately if you suspect it has leaked. Production API keys are issued separately and require manager approval.

Local development and review norms

Clone the monorepo and run ``make bootstrap`` to install toolchain dependencies and pre-commit hooks. The storage platform services run locally via the dev-cluster docker-compose file in `infra/dev-cluster`. Bring it up with ``make dev-up`` and tear it down with ``make dev-down``.

Integration tests against the dev cluster live in the `tests/integration` directory and take about ten minutes to run. Run the fast unit suite with ``make test`` before every push; the integration suite runs automatically in CI on each pull request.

Code review norms

Every change needs at least one approval from a team member other than the author. Storage-shard and replication code additionally requires sign-off from a senior engineer, since bugs there can affect data durability. Keep pull requests small — large, multi-purpose PRs take longer to review and are more likely to introduce regressions.

On-call rotation and support

New engineers join the on-call rotation after their first month, shadowing an experienced on-call engineer for one full rotation before taking a shift solo. Rotations are weekly and handoffs happen on Tuesdays at 10:00 in the team's local time zone.

If you are paged, acknowledge the alert within five minutes. Use the runbooks linked from each alert as your starting point — most incidents map to a documented procedure. If you are unsure how to proceed, escalate early by paging the secondary on-call rather than spending too long investigating alone.

Where to ask questions

Use the #storage-platform channel for general questions and #acs-oncall for anything urgent and production-affecting. There are no bad questions during your first few months — asking early is much cheaper than debugging a larger problem later.

What to read first

Start by reading the architecture overview, the API reference, and the onboarding runbook. Then review the storage shard service README and the shard manager design doc in docs/. Focus on the traffic flow: Gateway -> Shard Manager -> Storage Shard -> Metadata Store.

Key repos and services to know:

- `gateway-service`: request ingress, auth, rate limiting, and request routing.
- `shard-manager`: shard ownership, consistent hashing, and rebalancing.
- `storage-shard`: object storage, replication, and local disk durability.
- `metadata-store`: object metadata, listings, access control, and policy enforcement.

Understanding the separation between object bytes and metadata is critical: object bytes are stored on the shard cluster, while the metadata store is the source of truth for routing and permissions.

First tasks and learning goals

Your first week should include:

- verifying your laptop bootstrap and repo access,
- running the local dev cluster and a simple end-to-end test,
- reading the onboarding docs, architecture overview, and API reference,
- shadowing a code review and asking two clarifying questions about the current design.

During week two, try to contribute a small documentation fix or a low-risk reliability improvement. This helps you learn the codebase and the release process while giving you an early feedback cycle.

Team culture

We value clear communication, incremental improvements, and operational ownership. If you see a repeatedly confusing workflow, suggest a doc update or a tooling improvement rather than letting the pain persist.